

プログラム説明資料

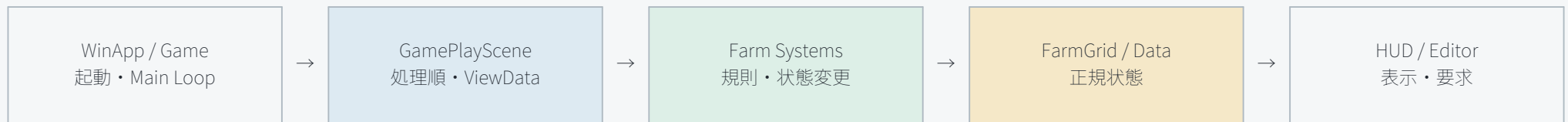
DirectX12 3D農業サンドボックスゲーム / 責務分離・状態遷移・安全性・用水路設計



対象コミット: a1d925e / 個人就職作品 / 2026年8月28日

01 全体アーキテクチャ

Gameは起動とメインループ、Sceneは進行統括、Systemは状態変更、UIは表示と入力通知だけを担当します。



Application	Game / SceneManager / GamePlayScene	起動、シーン切替、各Systemの処理順、ViewData作成
Farm Data	FarmGrid / FarmTypes / FarmRules	Tile配列、範囲検証、enum、設定値、派生状態
Farm Systems	ToolAction / Growth / Economy / Irrigation / Document	入力要求の検証、状態変更、Undo、保存、通水探索
Presentation	FarmHUD / FarmControllerWindow / FarmVisualSystem	読み取り専用Snapshot表示、操作要求の通知、簡易3D可視化
Engine	DirectXCommon / Object3d / Sprite / LineDrawer	DX12 Resource、Descriptor、Shader、描画コマンド

02 FarmGridと状態モデル

栽培状態と土地機能を分離し、将来の水シミュレーションを既存の栽培ループへ無理に混ぜない構造です。

FarmTileState

Empty / Tilled / Planted。耕作の状態遷移を表す。

FarmTileFeature

None / Canal / WaterSource。栽培とは直交する土地機能。

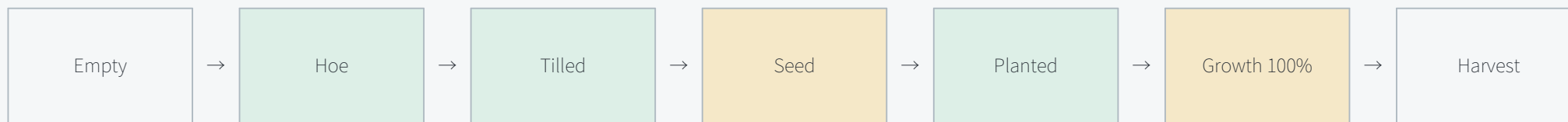
派生状態

GrowthStage、MoistureStatus、CanalSupply。再計算可能な値は保存しない。

主要データ

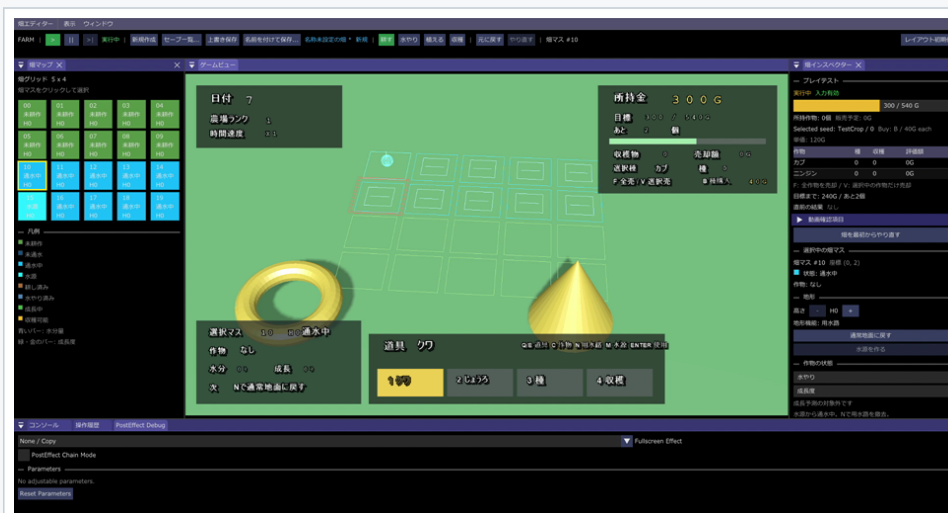
- heightLevel: H0～H2。変更時に範囲をClampし、通水探索を再実行。
- crop / moisture / growth: 作物種、正規化水分量、正規化成長度。非有限値と範囲外を拒否。
- feature: 水源または用水路。栽培データとの重複を禁止。
- selectedTile: indexと座標変換を一元化し、空GridとOOBは失敗値を返す。

状態遷移の例



03 用水路と通水判定

FarmIrrigationSystemはGridを変更せず、地形と土地機能から供給状態を導出します。



水源と用水路を配置。高さや接続状態をInspectorで確認。

計算量

O(tile count)。毎frameのfile IO、GPU allocation、Descriptor 確保は行わない。

保存方針

WaterSourceとCanalは保存。Supplied/Dryは派生状態なのでLoad後に再計算。

現在の制限

水量、流向、滞留、蒸発、土壌浸潤は未実装。表示はLineDrawerによる検証用。

処理順

1. 全Tileの供給状態を未供給で初期化。
2. WaterSourceを探索開始点としてQueueへ追加。
3. 4近傍のCanalだけを候補にする。
4. neighbor.height <= current.height の場合だけ供給。
5. 訪問済みを記録し、Grid全体を一度だけ処理。

04 作物・品質・経済

作物ごとの成長プロファイルと収穫品質を分離し、将来の作物追加とバランス調整に備えています。



品質5軸をレーダーチャートで可視化。

主要System

FarmGrowthSystem: 作物別の成長速度、水分閾値、収穫予測。

FarmQualitySystem: 大きさ・形・色・水分・成長の5軸評価。

FarmEconomySystem: 種購入、作物別在庫、選択販売、全販売。

FarmFeedbackSystem: 直近の購入・収穫・販売結果を境界付きで保持。



実行時安全性

- 未対応CropType、在庫0、価格0以下、不正なindexを状態変更前に拒否。
- 所持金加算はoverflowを確認し、失敗時は在庫を減らさない。
- 品質値は有限性と範囲を検証し、Undo時は収穫前の品質履歴と在庫価値まで復元。

05 Undo / RedoとFarm Document

操作履歴と永続化を別責務にし、編集操作とファイル復元のどちらでも複数Systemの整合性を守ります。

Command History

Tile変更、収穫、販売に必要なBefore/After Snapshotを保持。
適用失敗時は履歴位置を進めない。

Document Schema 4

Grid、feature、economy、seed inventory、selected crop、
last qualityをJSON保存。旧Schemaも読み込み可能。

Atomic Save

一時ファイルへ書き、成功後に置換。破損JSON、型不一致、
範囲外値を拒否。

Load処理



失敗時の扱い

各SystemのRestoreを個別に試行し、途中失敗では元SnapshotへRollbackします。短絡評価で復元処理が飛ばないようにし、失敗をUIへ通知します。保存対象ではない通水状態や成長ステージは、復元後の正規データから再計算します。

06 HUDとDebug Editor

プレイヤー向けHUDはSprite系、開発者向けInspectorはImGuiで分離しています。



中央Game View内がRuntime HUD。左右と下部はDebug Editor。

Runtime HUD

日本語ラベル、Day、所持金、選択Tile、道具、作物、在庫、品質、売却結果をSprite/BitmapFont系で表示。Releaseでも使用可能。

Debug Editor

Game Map、Inspector、品質レーダー、Save/Load、Undo/Redo、検証状態をImGuiで表示。ゲーム状態は直接書き換えず、typed requestをSceneへ通知。

可読性

情報を左上・右上・左下・下中央へ分散し、背景パネルと余白を固定。日本語の長さに合わせて領域を確保。

検証性

同じSnapshotをHUD、Map、Inspectorへ渡し、選択Tile・水分・成長・在庫・通水の不一致を発見しやすくする。

境界

UIに成長式、売価計算、通水探索、Save処理を置かない。UIは描画と入力通知だけ。

07 DirectX12境界と性能リスク

農業機能の追加で描画資源の所有権や同期規則を崩さないことを優先しています。

Resource / VRAM	Texture・Model・Sprite・LineDrawer側が所有。FarmはHandle／ViewDataのみ。	Asset unloadとgeneration管理の全経路適用。
Descriptor	上位ゲームロジックへDescriptor indexを公開しない。	動的Asset増加時のHeap枯渇監視。
Barrier / Fence	今回のFarm更新ではGPU Resource状態を変更しない。	水面描画追加時のRenderTarget／Compute同期。
CPU負荷	Grid更新と通水探索はO(tile count)。固定長配列と再利用vectorを使用。	Grid拡大時はdirty region化を検討。
DrawCall	検証用LineDrawerはTile単位。	完成描画ではinstancing／batchingが必要。
Allocation / IO	通常Update中にfile IOとGPU allocationを行わない。	Editor Asset importの非同期化。

08 検証状況と残課題

完成済みと未検証を明示し、採用資料で実装範囲を過大に見せないようにしています。

確認済み

Debug x64 build、起動smoke test、JSON schema、OOB／高低差通水のfocused test、HUD画像の表示確認。

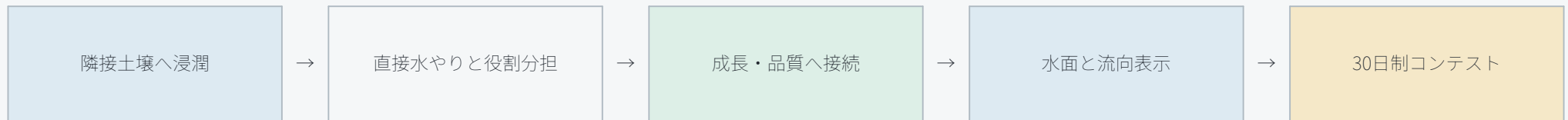
今回再確認するもの

Release x64、Release exe起動、Git由来ソースとResources、最終ZIP CRC、紹介動画再生。

未実装

水量保存、流向、土壌浸潤、蒸発、作物サイズへの最終影響、30日制コンテスト。

次の実装順



リポジトリ

https://github.com/YoshinoGento/CG2_00_01

本資料は個人就職作品ブランチのGitコミット a1d925e と、2026年8月28日に撮影した実行動画を基準に作成しています。